

PROJECT DOCUMENTATION

Personalized Networking Assistant

An AI-assisted web application for meaningful networking conversations

Prepared by Ravi Teja, Bhanu Prakash, Venkata Sai Mokshazna

Abstract

Networking events often require participants to start conversations with unfamiliar people while quickly understanding the event context. The Personalized Networking Assistant addresses this problem by accepting an event description and optional user interests, extracting prominent phrases, generating starter questions, retrieving a short factual summary, and retaining the interaction for later review.

The solution uses a Streamlit interface and a FastAPI backend. KeyBERT, initialized with a DistilBERT sentence-embedding model, identifies one- and two-word keyphrases from the event description. The first available topic drives three templated questions. A Wikipedia lookup supplies a short contextual summary, and JSON files store history and feedback.

Table of Contents

Section	Page
Abstract	2
Introduction	4
Problem Statement	5
Objectives and Scope	6
Technology Overview	7
Requirements	8
System Architecture	9
Module Design	10–11
Workflow	12
API Specification	13
User Interface	14
Data Management and Security	15
Algorithms	16
Testing	17
Results and Limitations	18
Deployment	19
Future Enhancements	20
Conclusion and References	21

1. Introduction

Purpose and context

Networking is more effective when participants arrive with a clear understanding of the event themes and a few respectful questions prepared. In practice, event descriptions can be broad, and a person may have little time to research speakers, topics, or industry vocabulary before attending.

The Personalized Networking Assistant is a lightweight support tool for this preparation stage. A user describes an event and supplies an interests field in the web interface. The application extracts candidate topics and produces an approachable question set around the strongest topic. It also provides a concise Wikipedia-based fact check to help the user enter the discussion with context.

- Reduce the friction of preparing for a networking event.
- Turn unstructured event text into visible, reusable topic cues.
- Keep the interaction simple enough for a short pre-event session.

2. Problem Statement

People attending professional events may struggle to convert a generic event announcement into an engaging opening conversation. Searching manually for topic ideas can be slow, and generic prompts may feel disconnected from the event itself. A helpful assistant should extract themes directly from the supplied description, offer neutral questions, and avoid forcing the user through a complex workflow.

Challenge	Project response
Unstructured event text	KeyBERT extracts up to five relevant keyphrases.
Uncertainty about what to ask	A question template creates three starters from the top topic.
Need for quick context	Wikipedia lookup returns a short summary when a search succeeds.
Need to revisit output	History records the event, topics, conversation, and fact result.
Need to signal usefulness	Like/Dislike feedback is persisted for later analysis.

3. Objectives and Scope

The project is designed as a functional prototype of an AI-assisted networking preparation tool. Its objectives are focused on a small, end-to-end user journey rather than a fully personalized recommendation platform.

Primary objectives

- Accept a natural-language description of a networking event.
- Extract interpretable candidate topics from the description.
- Generate a short set of topic-centered conversation starters.
- Retrieve one concise factual context result.
- Record interactions and collect feedback.
- Expose the workflow through a simple browser interface and REST API.

Current scope boundaries

- No account system, authentication, or role management.
- No relational database; persistence uses local JSON files.
- No direct use of the entered interests value in the generation algorithm.
- No guarantee that external Wikipedia content is complete, current, or suitable for every event.

4. Technology Overview

Technology	Role in the implementation
Python	Primary language for the frontend, API, NLP integration, storage, and tests.
FastAPI	Defines REST routes, validates request bodies with Pydantic, and returns JSON.
Streamlit	Provides the browser-based form, output sections, feedback buttons, and history view.
KeyBERT	Extracts ranked keyphrases from event text.
DistilBERT sentence model	Supplies sentence embeddings to the KeyBERT model.
Wikipedia library	Searches Wikipedia and requests a short summary for factual context.
Pytest / TestClient	Supports automated endpoint checks without starting a separate server.

The requirements file pins these dependencies and their transitive packages. The architecture intentionally keeps the UI separate from backend logic, allowing either side to be changed independently.

5. Requirements

Functional requirements

1. Accept event and interests text from a user.
2. Return detected topics, a conversation starter, and a fact result.
3. Save a record for each generation request.
4. Receive Like or Dislike feedback for an event.
5. Return saved conversation history.

Non-functional requirements

Area	Expectation
Usability	The primary workflow should be understandable without a manual.
Modularity	Topic extraction, generation, fact lookup, history, and feedback remain separate modules.
Resilience	External lookup failures should return a readable result instead of crashing the route.
Testability	Routes should be callable through FastAPI TestClient.
Portability	The project should run from a Python environment with the listed packages.

6. System Architecture

The application uses a simple client-server arrangement. Streamlit collects input and calls the locally hosted FastAPI service. The API coordinates the NLP, generation, fact lookup, and persistence modules before returning a JSON response to the interface.

Layer	Component	Responsibility
Presentation	app.py / Streamlit	Collect event input; display topics, starter, fact, feedback, and history.
API	backend/main.py	Validate request schemas and orchestrate application services.
NLP	event_analyzer.py	Use KeyBERT + DistilBERT embeddings to extract keyphrases.
Content	topic_generator.py; fact_checker.py	Produce templated questions and Wikipedia contextual summaries.
Persistence	history_logger.py; feedback_logger.py	Append JSON records and load history.
Quality	tests/test_api.py	Check core API routes and response fields.

Data flow: Event text → API validation → topic extraction → question generation + fact lookup → history save → JSON response → Streamlit display.

7. Module Design I

API and topic extraction

7.1 FastAPI service (backend/main.py)

The service defines two Pydantic request models: `EventData` with event and interests strings, and `FeedbackData` with event and feedback strings. The route handlers are small orchestration functions; their main value is keeping HTTP concerns separate from processing logic.

Route	Method	Behavior
/	GET	Returns a welcome message.
/generate	POST	Extracts topics, builds a starter, retrieves a fact, saves history, and returns JSON.
/feedback	POST	Saves a feedback record and returns a confirmation message.
/history	GET	Returns the contents of the history JSON file.

7.2 Event analysis (backend/event_analyzer.py)

A KeyBERT model is created with the `distilbert-base-nli-mean-tokens` embedding model. When `analyze_event` receives text, it asks KeyBERT for up to five one- or two-word phrases, removes English stop words, and returns only the keyphrase text (not the scores).

8. Module Design II

Generation, facts, and persistence

8.1 Conversation generation

`generate_conversation` selects the first detected topic, or the fallback topic “technology” when no topics exist. It returns three open-ended questions. Because the output is templated, the questions are predictable and immediately understandable; however, they do not yet reflect the user’s interests or a topic ranking threshold.

8.2 Fact checking

`fact_check` searches Wikipedia using the supplied value and requests a two-sentence summary from the first result. It handles disambiguation and missing-page conditions with friendly messages. Other exceptions are currently returned as their string representation, which is useful for diagnosis but should be refined for production.

8.3 JSON persistence

File	Record fields	Use
data/history.json	event, topics, conversation, fact	Stores each generation result.
data/feedback.json	event, feedback	Stores a Like or Dislike response.

9. End-to-End Workflow

6. The user enters an event description and optional interests in the Streamlit page.
7. The Generate Conversation button sends a POST request to `http://127.0.0.1:8000/generate`.
8. FastAPI validates the JSON payload as `EventData`.
9. The event analyzer extracts up to five topics.
10. The generator creates three prompts from the first topic, and the fact checker requests context.
11. The history logger appends a full result record to `data/history.json`.
12. FastAPI returns topics, conversation, and fact as JSON.
13. Streamlit displays the result and allows feedback; View History requests `/history` and renders saved records.

Important implementation detail: Although the frontend submits interests, the current backend does not pass it into `analyze_event` or `generate_conversation`. This is a clear path for future personalization.

10. API Specification

POST /generate

Request body:

```
{
  "event": "Artificial Intelligence in Healthcare",
  "interests": "Machine Learning"
}
```

Successful response shape:

```
{
  "topics": ["artificial intelligence", "healthcare"],
  "conversation": "1. What inspired you to work in ...?",
  "fact": "A short Wikipedia summary or a readable status message"
}
```

POST /feedback

```
{ "event": "Artificial Intelligence in Healthcare", "feedback": "Like" }
```

GET /history

Returns an array of previously stored history records. The route does not currently paginate results or filter by user, date, or event.

11. User Interface Design

The Streamlit interface intentionally uses a linear interaction model. The page title identifies the assistant, a text area asks for the event description, and a text input accepts user interests. The Generate Conversation button initiates the API call.

UI element	User purpose	Current behavior
Event Description	Provide event context	Required in practice for meaningful extraction; no frontend validation is implemented.
Interests	State personal interests	Sent to the API but not used by backend logic.
Generate Conversation	Create assistance	Displays topics, starter questions, fact result, and feedback controls.
Like / Dislike	Rate usefulness	Posts an event-feedback pair to /feedback.
View History	Review prior output	Fetches and displays every saved history record.

The UI can be improved with loading states, error handling around failed requests, input validation, and structured display cards. These are enhancements, not existing behavior.

12. Data Management and Security Considerations

The application stores data in repository-local JSON files. This makes the prototype easy to inspect and run, but it is not appropriate for a multi-user production deployment. Concurrent writes, file growth, backup procedures, and access control are not managed by the current implementation.

Area	Current state	Recommended direction
Persistence	Plain JSON append/read	Use a database with migrations, indexes, and retention rules.
Identity	No authentication	Add secure account or organization identity if records become personal.
Input handling	Pydantic type validation	Add lengths, empty-value validation, and rate limits.
External content	Wikipedia first search result	Show source attribution and use safe user-facing errors.
Secrets	None shown in source	Keep future API keys in environment variables, never source files.

Privacy: Event descriptions and feedback can reveal professional interests. A production release should define consent, retention, deletion, and access policies before collecting user data at scale.

13. Algorithms and Processing Logic

Topic extraction pseudocode

```
keywords = KeyBERT.extract_keywords(  
    event, keyphrase_ngram_range=(1, 2),  
    stop_words="english", top_n=5  
)  
topics = [keyword for keyword, score in keywords]
```

KeyBERT ranks candidate phrases based on embeddings. The configured n-gram range allows both single terms and two-word phrases, which is useful for event themes such as “machine learning” or “healthcare innovation.” Scores are generated internally but are not returned by the project code.

Conversation selection logic

```
topic = topics[0] if topics else "technology"  
return three templated questions using topic
```

This fallback makes the generator resilient when no topics are found, but it can make the result generic. A future version could rank topics against interests, diversify question types, and use an LLM with guardrails.

14. Testing Strategy and Current Tests

The repository includes Pytest configuration and an API test module using FastAPI TestClient. The tests exercise the service in-process, making them fast and suitable for basic regression checks.

Test	What it checks	Observation
test_home	GET / returns HTTP 200	Confirms the application is reachable.
test_generate	POST /generate returns HTTP 200	Confirms the primary route can process a representative event.
Response field checks	topics, conversation, and fact exist	Checks response shape, not content quality.

Recommended additional tests

- Mock KeyBERT and Wikipedia to avoid slow model/network dependencies.
- Test empty and very long input, network failures, and invalid JSON payloads.
- Verify JSON persistence behavior and corrupted-file recovery.
- Assert that conversation output includes a selected topic.
- Add UI-level tests or at minimum client request error handling tests.

15. Results, Strengths, and Limitations

Observed strengths

- Clear separation of responsibilities across small backend modules.
- A complete flow from event input to topics, prompts, facts, history, and feedback.
- Readable API surface with only four routes.
- Graceful handling for common Wikipedia disambiguation and missing-page cases.

Known limitations

- The interests field is unused, so the system is not yet genuinely personalized.
- Conversation starters are fixed templates and may be repetitive.
- Fact lookup searches using the entire event description rather than a selected topic.
- Plain JSON persistence is vulnerable to concurrent-write and scaling issues.
- There is no frontend error handling for unavailable backend or malformed responses.
- No formal accuracy, relevance, latency, or user-satisfaction evaluation is included in the repository.

16. Deployment and Operation Guide

The backend and frontend are started as separate processes. The API is expected at port 8000 and the Streamlit application sends requests to the hard-coded local address `http://127.0.0.1:8000`.

Setup steps

14. Create and activate a Python virtual environment.
15. Install packages with `pip install -r requirements.txt`.
16. Start the API with `uvicorn backend.main:app --reload`.
17. In a second terminal, start the interface with `streamlit run app.py`.
18. Open the Streamlit URL shown in the terminal and submit an event description.
19. Run `pytest` to execute the included test suite.

Operational notes

- The embedding model may download or initialize on first use, which can delay the first request.
- Wikipedia lookups depend on network availability.
- The data directory must be writable for history and feedback storage.

17. Future Enhancements

Priority	Enhancement	Expected value
High	Use interests in topic ranking and prompt selection	Makes the assistant actually personalized.
High	Add robust request and error handling in Streamlit	Improves user trust when the service or network fails.
High	Replace JSON files with a database	Supports concurrent users, filtering, and reliable persistence.
Medium	Use selected topic for fact lookup and show source links	Improves factual relevance and transparency.
Medium	Add authentication and user-specific history	Enables privacy controls and tailored saved results.
Medium	Add LLM-based prompt variations with safety controls	Creates more natural, diverse questions.
Low	Add analytics dashboard for feedback trends	Helps evaluate usefulness over time.

Any expansion that uses generative AI or stores personal data should be paired with clear policies, safe fallbacks, evaluation criteria, and monitoring.

18. Conclusion and References

The Personalized Networking Assistant is a well-scoped prototype that demonstrates an end-to-end AI-assisted workflow for networking preparation. It combines keyphrase extraction, templated prompt creation, external factual context, persistent history, feedback collection, a REST API, and a simple web interface.

Its strongest next step is to connect the user interests input to the topic and prompt selection process. With better error handling, database-backed storage, source transparency, and meaningful evaluation, the prototype can evolve into a more robust and genuinely personalized assistant.

References

- Project source files: `app.py`; `backend/main.py`; `backend/event_analyzer.py`; `backend/topic_generator.py`; `backend/fact_checker.py`; `backend/history_logger.py`; `backend/feedback_logger.py`; `tests/test_api.py`.
- FastAPI documentation: <https://fastapi.tiangolo.com/>
- Streamlit documentation: <https://docs.streamlit.io/>
- KeyBERT project: <https://maartengr.github.io/KeyBERT/>
- Wikipedia Python library: <https://pypi.org/project/wikipedia/>

End of report.